You're now thinking like a systems architect—which is exactly what this business needs. Let's turn your stack into a **real, production-grade, AI-assisted architecture** that can actually ship.

I'll give you:

1. □ **Final Combined Architecture (with n8n as the brain)**
2. □ **Autonomous AI Agent Prompts (build + test + deploy)**
3. 📱 **Mobile Apps (iOS + Android with QR/Barcode scanning)**
4. 🔄 **End-to-End Workflow (parcel scan → data → tracking)**

□ 1. FINAL ROBUST ARCHITECTURE (HYBRID + AUTOMATION-FIRST)

🎯 Philosophy

- Odoo = business backbone
- Directus/Strapi = API layer
- n8n = automation brain □
- Custom services = competitive edge
- Mobile apps = operational power

🧱□ SYSTEM ARCHITECTURE

```
[ Mobile Apps (iOS/Android) ]
        ↓
[ Next.js Web App ]
        ↓
[ API Layer (Directus / Strapi) ]
        ↓
[ Core Services Layer ]
   - Pricing Engine
   - Booking Engine
   - Tracking Engine
        ↓
[ PostgreSQL / Supabase ]
        ↓
[ Odoo ERP ]
        ↓
[ OpenTMS ]
        ↓
[ n8n Automation Layer ]
        ↓
[ External Systems ]
   - Airlines
   - Email / WhatsApp
```

```
- Customs
```

🔥 ROLE OF n8n (CRITICAL)

n8n becomes your **central nervous system**:

Automations:

- Booking → send airline request
- Shipment created → notify customer
- Status update → trigger tracking update
- Invoice generated → send payment link
- QR scan → fetch shipment instantly

□ EVENT-DRIVEN FLOW (KEY IDEA)

Everything is event-based:

```
User books shipment →
Webhook →
n8n →
Updates DB →
Triggers Odoo →
Sends notification →
Updates tracking
```

□ 2. AUTONOMOUS AI AGENTS (PROMPTS)

You can use tools like:

- OpenAI API
- LangChain

□💻 AGENT 1: SYSTEM ARCHITECT

Prompt:

```
You are a senior software architect.

Design a scalable logistics platform using:
- Odoo (ERP)
```

```
- Directus (API layer)
- PostgreSQL
- n8n (automation)
- Next.js frontend

Requirements:
- Multi-country (Bangladesh + Malaysia)
- Air cargo booking system
- Real-time tracking
- Role-based access (admin, shipper, operator)

Output:
- System diagram
- Service breakdown
- API structure
- Database schema
```

⚙□ AGENT 2: BACKEND BUILDER

```
You are a backend engineer.

Build a logistics backend with:
- Node.js (NestJS)
- PostgreSQL

Modules:
- Pricing engine (weight, CBM, margin)
- Booking system
- Shipment tracking
- QR/barcode lookup

Integrate:
- Directus API
- Odoo (REST/XMLRPC)

Output:
- Clean modular code
- Docker setup
- API endpoints
```

🔄 AGENT 3: n8n AUTOMATION ENGINEER

```
You are an n8n expert.

Create workflows for:
1. New booking → notify airline via email
2. Shipment status update → notify customer
3. Invoice created in Odoo → send payment link
4. QR scan → fetch shipment details
5. Failed shipment → alert admin

Use:
- Webhooks
- Email nodes
- HTTP requests
```

```
Output:
- n8n workflow JSON
- Setup instructions
```

📱 AGENT 4: MOBILE APP BUILDER

```
You are a mobile app developer.

Build a cross-platform app using:
- React Native (Expo)

Features:
- Login (JWT)
- Scan QR/barcode
- Fetch shipment details
- Update shipment status
- Offline scan support

Use:
- Camera API
- Barcode scanning library

Output:
- Full app code
- API integration
- Build instructions for iOS & Android
```

□ AGENT 5: QA + TESTING

```
You are a QA engineer.

Test:
- Booking flow
- Pricing accuracy
- QR scan retrieval
- API performance
- Mobile app usability

Output:
- Test cases
- Bug reports
- Load testing results
```

🚀 AGENT 6: DEVOPS / DEPLOYMENT

```
You are a DevOps engineer.

Deploy system using:
- AWS
- Docker
- CI/CD (GitHub Actions)

Services:
```

```
- Backend API
- n8n
- Odoo
- PostgreSQL

Output:
- Deployment scripts
- Environment configs
- Scaling strategy
```

📱 3. MOBILE APPS (iOS + ANDROID)

Tech Stack

- React Native (Expo)

📷 QR + BARCODE SYSTEM

Libraries:

- expo-barcode-scanner
- react-native-camera

🎁 WHAT QR/Barcode DOES

Each shipment gets:

- QR code
- Barcode (Code128)

📲 Scan Flow

```
User scans code →
App calls API →
Returns shipment data →
Display:
  - Sender
  - Destination
  - Status
  - Weight
```

🔥 Example API

```
GET /shipment/{tracking_id}
```

□ Smart Feature

- Offline scan → sync later via n8n
- Auto status update (Delivered, In Transit)

🔄 4. REAL WORKFLOW (END-TO-END)

🎁 Shipment Lifecycle

Step 1: Booking

- User books via web/app

Step 2: n8n triggers

- Sends email to airline
- Updates Odoo

Step 3: QR generated

- Stored in DB
- Sent to user

Step 4: At warehouse

- Staff scans QR
- Status → “Received”

Step 5: At airport

- Scan again → “In Transit”

Step 6: Delivery

- Final scan → “Delivered”

🔐 SECURITY LAYER

- JWT authentication
- Role-based access
- API rate limiting

⚡ FINAL STACK SUMMARY

Layer	Tech
ERP	Odoo
API	Directus
Automation	n8n
Frontend	Next.js
Mobile	React Native
DB	PostgreSQL / Supabase

🚨 FINAL STRATEGIC INSIGHT

This architecture gives you:

✅ Fast MVP (6–10 weeks)
✅ Low cost
✅ Scalable to airline level later
✅ Automation-first (small team can run it)

Perfect—now we're moving into **build-ready assets**. I'll give you production-grade starting points you can directly use or hand to developers.

🔄 1. n8n WORKFLOW JSON (REAL EXAMPLE)

🎁 Workflow: Booking → Notify Airline → Update System

You can import this directly into n8n

```
{
  "name": "New Booking Workflow",
  "nodes": [
    {
      "parameters": {
        "path": "new-booking",
        "httpMethod": "POST",
        "responseMode": "onReceived"
      },
      "name": "Webhook",
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 1,
      "position": [250, 300]
    },
    {
      "parameters": {
        "functionCode": "return [{ json: { booking_id:
$json.body.booking_id, email: $json.body.email, weight: $json.body.weight }
}];"
      },
      "name": "Format Data",
      "type": "n8n-nodes-base.function",
      "position": [450, 300]
    },
    {
      "parameters": {
        "url": "https://your-api.com/shipments",
        "method": "POST",
        "jsonParameters": true,
        "options": {},
        "bodyParametersJson": "={{$json}}"
      },
      "name": "Create Shipment",
      "type": "n8n-nodes-base.httpRequest",
      "position": [650, 300]
    },
    {
      "parameters": {
        "fromEmail": "ops@yourcompany.com",
        "toEmail": "cargo@airline.com",
        "subject": "New Cargo Booking",
        "text": "New booking received: {{$json.booking_id}}"
      },
      "name": "Send Email to Airline",
      "type": "n8n-nodes-base.emailSend",
      "position": [850, 200]
    },
    {
      "parameters": {
        "fromEmail": "ops@yourcompany.com",
        "toEmail": "={{$json.email}}",
        "subject": "Booking Confirmation",
        "text": "Your shipment is booked successfully."
      },
      "name": "Notify Customer",
      "type": "n8n-nodes-base.emailSend",
      "position": [850, 400]
    }
  ],
  "connections": {
    "Webhook": {
```

```
      "main": [[{ "node": "Format Data", "type": "main", "index": 0 }]]
    },
    "Format Data": {
      "main": [[{ "node": "Create Shipment", "type": "main", "index": 0 }]]
    },
    "Create Shipment": {
      "main": [
        [{ "node": "Send Email to Airline", "type": "main", "index": 0 }],
        [{ "node": "Notify Customer", "type": "main", "index": 0 }]
      ]
    }
  }
}
```

🗄 □ 2. DATABASE SCHEMA (CORE TABLES)

Use PostgreSQL / Supabase.

👤 users

```
id (uuid, pk)
name
email (unique)
password_hash
role (admin, shipper, operator)
created_at
```

🎁 shipments

```
id (uuid, pk)
tracking_number (unique)
user_id (fk)
origin
destination
status (pending, booked, in_transit, delivered)
weight
volume
price
created_at
```

📊 rates

```
id (uuid, pk)
airline
origin
```

```
destination
rate_per_kg
min_charge
currency
effective_date
```

📚 bookings

```
id (uuid, pk)
shipment_id (fk)
status (pending, confirmed, cancelled)
airline
booking_reference
created_at
```

📍 tracking_events

```
id (uuid, pk)
shipment_id (fk)
status
location
timestamp
```

📎 documents

```
id (uuid, pk)
shipment_id (fk)
file_url
type (invoice, airway_bill)
uploaded_at
```

▢ qr_codes

```
id (uuid, pk)
shipment_id (fk)
qr_data
barcode_data
generated_at
```

🔌 3. API SPEC (SWAGGER STYLE)

You can plug this into Swagger UI.

Base URL

`https://api.yourcompany.com`

🔐 Auth

POST /auth/login

```
{
  "email": "user@email.com",
  "password": "password"
}
```

🎁 Create Shipment

POST /shipments

```
{
  "origin": "DAC",
  "destination": "LHR",
  "weight": 120,
  "volume": 1.5
}
```

🎁 Get Shipment

GET /shipments/{id}

Response:

```
{
  "tracking_number": "ABC123",
  "status": "in_transit",
  "origin": "DAC",
  "destination": "LHR"
}
```

📍 Tracking

GET /tracking/{tracking_number}

```
[
  {
    "status": "Received",
    "location": "Dhaka",
    "timestamp": "2026-03-29T10:00:00Z"
```

```
  }
]
```

💰 Pricing

POST /pricing/calculate

```
{
  "origin": "DAC",
  "destination": "LHR",
  "weight": 100
}
```

📷 QR Lookup

GET /scan/{code}

```
{
  "shipment_id": "uuid",
  "status": "in_transit",
  "details": "Full shipment info"
}
```

📂 4. GITHUB REPOS TO FORK (HIGH VALUE)

Here are real, useful starting points:

□ ERP / Backend

- Odoo
  ☞ https://github.com/odoo/odoo
- ERPNext
  ☞ https://github.com/frappe/erpnext

🔌 API Layer

- Directus
  ☞ https://github.com/directus/directus

- Strapi
  ☞ https://github.com/strapi/strapi

🔄 Automation

- n8n
  ☞ https://github.com/n8n-io/n8n

📱 Mobile (QR + Barcode ready)

- React Native
  ☞ https://github.com/expo/expo

QR scanning:
☞ https://github.com/expo/expo/tree/main/packages/expo-barcode-scanner

---

🚚 Logistics / TMS

- OpenTMS
  ☞ (older but useful reference)
  https://sourceforge.net/projects/opentms/

□ AI Agents / Automation

- LangChain
  ☞ https://github.com/langchain-ai/langchain

🚀 FINAL BUILD STRATEGY (IMPORTANT)

Week 1–2

- Deploy Directus + DB

- Setup n8n

Week 3–4

- Build API + pricing engine
- Import rates manually

Week 5–6

- Mobile app (QR scanning)
- Booking flow

Week 7–8

- Integrate Odoo
- Automate workflows